



# 2025/08/12

☰ 預計完成

ULIP測試結果

3D 多模態檢索實驗，跑六種檢索方向的評估。因為我要確保 PointCloud、Text、Image 三個模態彼此之間的檢索都能對齊，並記錄在wandb

## 1) 前向抽三模態向量 (Text / PointCloud / Image)

```
all_pc, all_txt, all_img = [], [], []
all_tax, all_md1 = [], []

for batch in loader:
    taxonomy_ids = batch[0]
    model_ids = batch[1]
    txt_tokens = batch[2].to(device).long()
    pc = batch[3].to(device).float()
    img = batch[4].to(device)

    out = model(pc, txt_tokens, img)
    pc_emb = F.normalize(out["pc_embed"], dim=-1)
    txt_emb = F.normalize(out["text_embed"], dim=-1)
    img_emb = F.normalize(out["image_embed"], dim=-1)

    all_pc.append(pc_emb.cpu())
    all_txt.append(txt_emb.cpu())
    all_img.append(img_emb.cpu())
    all_tax.extend(taxonomy_ids)
    all_md1.extend(model_ids)

pc_feats = torch.cat(all_pc, dim=0)
txt_feats = torch.cat(all_txt, dim=0)
img_feats = torch.cat(all_img, dim=0)
```

逐批前向，模型必須回傳三個鍵：`pc_embed / text_embed / image_embed`。

對三個嵌入做 L2 正規化 (`F.normalize`)。這樣後面用內積 `q @ g.T` 就是 cosine，相似度計算變成單純矩陣乘法，速度快。把每批的向量搬回 CPU、最後用 `torch.cat` 串成完整驗證集的特徵矩陣。`all_tax / all_md1` 同步蒐集對應的 ID。

## 2) 建立物件 ID 對索引的對照 (正例集合)

```
object_ids = [f"{t}-{m}" for t, m in zip(all_tax, all_md1)]
from collections import defaultdict
obj2idxs = defaultdict(list)
for i, oid in enumerate(object_ids):
    obj2idxs[oid].append(i)
```

把 `(taxonomy_id, model_id)` 串成 `object_id`，例如 `"chair-1234"`。`obj2idxs[oid]` 收集同一物件在整個驗證集中的索引位置。後面排名時，我會把這些索引視為同一查詢的**全部正例**。

## 3) 取得「最佳名次」(Best Rank)

```
def _best_ranks(query_feats, gallery_feats, name):
    N = query_feats.size(0)
    ranks = []
    for i in range(0, N, chunk_size):
        j = min(i + chunk_size, N)
        sims = query_feats[i:j].to(device) @ gallery_feats.to(device).t()
        sorted_idx = torch.argsort(sims, dim=1, descending=True)
        for local in range(j - i):
            gid = i + local
            oid = object_ids[gid]
            pos_idxes = obj2idxs[oid]
            row = sorted_idx[local]
            best = float("inf")
            for p in pos_idxes:
                pos = (row == p).nonzero(as_tuple=True)[0]
                if pos.numel() > 0:
                    r = int(pos.item()) + 1 # 1-based
                    if r < best:
                        best = r
            ranks.append(best)
    return ranks
```

上面程式做了：

- 用分塊處理查詢，避免一次算 撐爆顯存。
- `sims = Q @ GT` 因為向量已做 L2 normalize，這就是 cosine 相似度。取得每個查詢的排序後，我會在該查詢的**所有正例位置**裡，挑出排最前面的那個名次當作結果（1-based rank）。

這樣同一物件若有多個樣本，只要其中一個排得前，就能反映在評分上。

#### 4) 摘要統計（MRR、R@k、平均/中位數名次）

```
def _summarize(ranks):
    valid = [r for r in ranks if r != float("inf")]
    if len(valid) == 0:
        return dict(mrr=0., r1=0., r5=0., r10=0., mean=0., median=0., n=len(ranks))
    r = torch.tensor(valid, dtype=torch.float32)
    return dict(
        mrr=(1.0/r).mean().item(),
        r1=(r<=1).float().mean().item(),
        r5=(r<=5).float().mean().item(),
        r10=(r<=10).float().mean().item(),
        mean=r.mean().item(),
        median=r.median().item(),
        n=len(valid)
    )
```

只對「有正例被找到」的查詢計算統計；找不到（`inf`）的不納入平均，避免扭曲。

公式

- MRR:  $\frac{1}{N} \sum 1/r_i$
- R@k:  $\sum \mathbf{1}[r_i \leq k]$
- Mean/Median: 名次的平均與中位數

#### 5) NDCG@k（多正例的前段排序品質）

```

def _ndcg_at_k(query_feats, gallery_feats, k=5):
    N = query_feats.size(0)
    scores = []
    for i in range(0, N, topk_chunk):
        j = min(i + topk_chunk, N)
        sims = query_feats[i:j].to(device) @ gallery_feats.to(device).t()
        topk = torch.topk(sims, k=min(k, sims.size(1)), dim=1, largest=True, sorted=True).
indices
        for row_idx in range(j - i):
            gid = i + row_idx
            oid = object_ids[gid]
            pos = set(obj2idxs[oid])
            if len(pos) == 0:
                scores.append(0.0); continue
            dcg = 0.0
            for rank_pos in range(topk.size(1)): # 0-based
                idx = int(topk[row_idx, rank_pos])
                if idx in pos:
                    dcg += 1.0 / np.log2(rank_pos + 2)
            P = min(len(pos), k)
            idcg = sum(1.0 / np.log2(t + 2) for t in range(P))
            scores.append(dcg / idcg if idcg > 0 else 0.0)
    return float(np.mean(scores))

```

只看前  $k$  名，用  $\text{topk}$  取排序清單。相關性設為二元：同物件=1，其餘=0。

- DCG :  $\sum_{j=1}^k \frac{rel_j}{\log_2(j+1)}$
- IDCG : 把正例全部排前面得到的理想值；最後  $NDCG = DCG/IDCG$ 。

## 6) 六向檢索配對與彙總輸出

```

pairs = [
    ("s2t", pc_feats, txt_feats),
    ("t2s", txt_feats, pc_feats),
    ("s2i", pc_feats, img_feats),
    ("i2s", img_feats, pc_feats),
    ("t2i", txt_feats, img_feats),
    ("i2t", img_feats, txt_feats),
]
final = {}
for tag, q, g in pairs:
    ranks = _best_ranks(q, g, tag.upper())
    s = _summarize(ranks)
    ndcg5 = _ndcg_at_k(q, g, k=k_ndcg)
    final[f"retr/{tag}_mrr"] = s["mrr"]
    final[f"retr/{tag}_r1"] = s["r1"]
    final[f"retr/{tag}_r5"] = s["r5"]
    final[f"retr/{tag}_r10"] = s["r10"]
    final[f"retr/{tag}_ndcg5"] = ndcg5
    final[f"retr/{tag}_mean"] = s["mean"]
    final[f"retr/{tag}_median"] = s["median"]

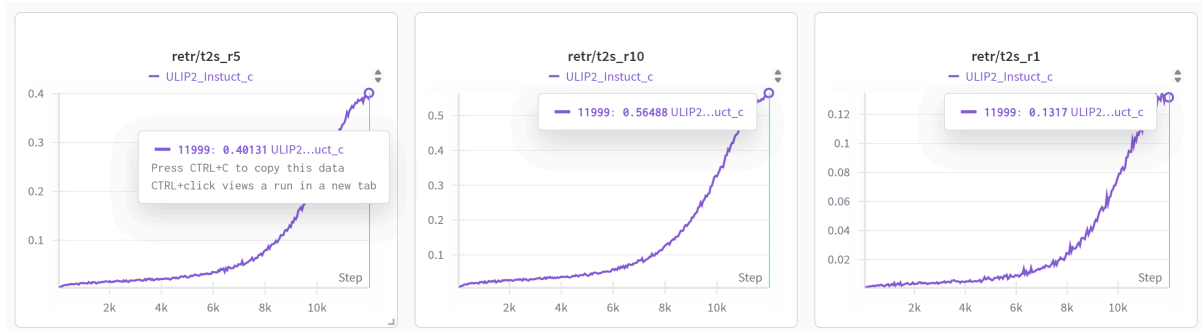
return final

```

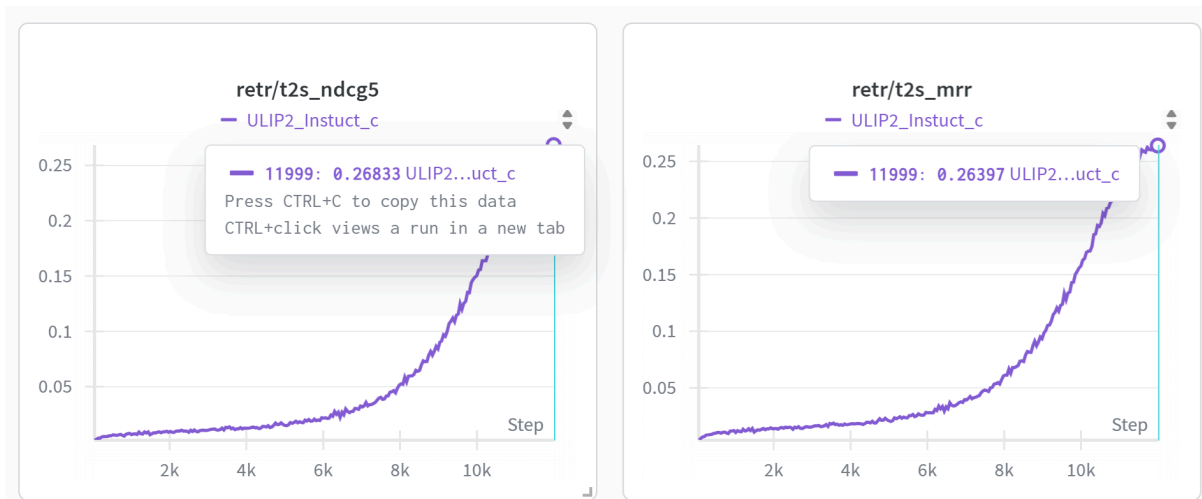
每個方向都算：MRR、R@1/5/10、NDCG@k、平均名次、名次中位數。用 `retr/{tag}_*` 命名結果欄位，回傳統一字典，方便 logger/可視化。

## 文字 → 3D

RR@K

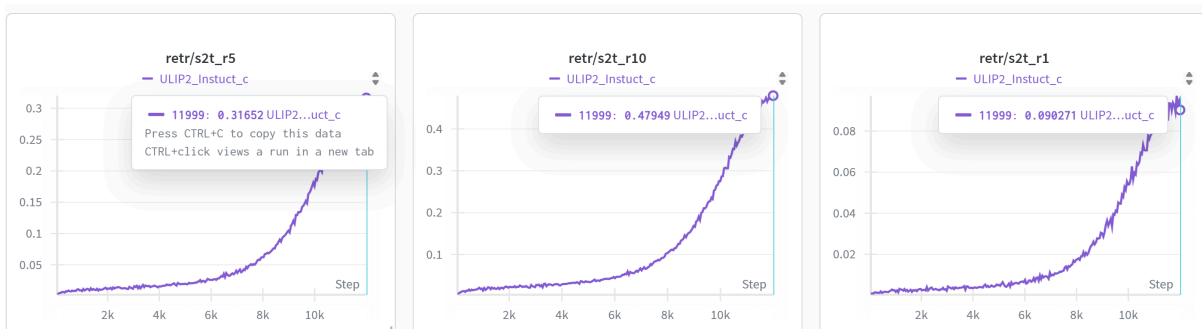


NDCG & MRR

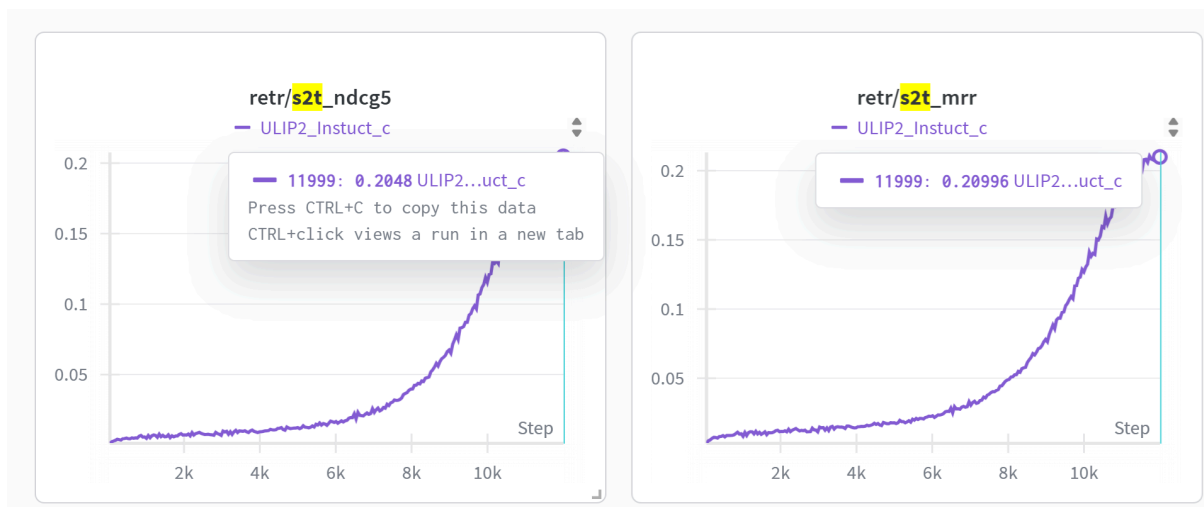


## 3D → 文字

RR@K



NDCG & MRR



## 測試

### 統計

S2T (PC→Text) retrieval completed: 14966 queries  
 S2T (PC→Text) Valid 14966/14966 Mean 35.7 Median 11.0  
 S2T (PC→Text) Rank≤1 1437 (9.6%) ≤5 4826 (32.2%) ≤10 7193 (48.1%) ≤100 14007 (93.6%)

T2S (Text→PC) retrieval completed: 14966 queries  
 T2S (Text→PC) Valid 14966/14966 Mean 25.6 Median 8.0  
 T2S (Text→PC) Rank≤1 2020 (13.5%) ≤5 5940 (39.7%) ≤10 8364 (55.9%) ≤100 14280 (95.4%)

S2I (PC→Image) retrieval completed: 14966 queries  
 S2I (PC→Image) Valid 14966/14966 Mean 10.3 Median 3.0  
 S2I (PC→Image) Rank≤1 5048 (33.7%) ≤5 10029 (67.0%) ≤10 11894 (79.5%) ≤100 14763 (98.6%)

I2S (Image→PC) retrieval completed: 14966 queries  
 I2S (Image→PC) Valid 14966/14966 Mean 7.3 Median 2.0  
 I2S (Image→PC) Rank≤1 6208 (41.5%) ≤5 11230 (75.0%) ≤10 12862 (85.9%) ≤100 14855 (99.3%)

### 表格

#### 檢索評估指標 (百分比)

| 檢索方向           | MRR (%) | R@1 (%) | R@5 (%) | R@10 (%) | nDCG@5 (%) |
|----------------|---------|---------|---------|----------|------------|
| S2T (PC→Text)  | 21.54   | 9.60    | 32.25   | 48.06    | 21.09      |
| T2S (Text→PC)  | 26.66   | 13.50   | 39.69   | 55.89    | 26.89      |
| S2I (PC→Image) | 48.76   | 33.73   | 67.01   | 79.47    | 51.34      |
| I2S (Image→PC) | 56.42   | 41.48   | 75.04   | 85.94    | 59.46      |

檢索效能對比（導入文字端 RAG 前後）

| MRR (%) Before | MRR (%) After | Δ     | nDCG@5 (%) Before | nDCG@5 (%) After | Δ     |
|----------------|---------------|-------|-------------------|------------------|-------|
| 26.66          | 29.00         | +2.34 | 26.89             | 29.50            | +3.61 |

| R@1 (%) Before | R@1 (%) After | Δ     | R@5 (%) Before | R@5 (%) After | Δ     | R@10 (%) Before | R@10 (%) After |
|----------------|---------------|-------|----------------|---------------|-------|-----------------|----------------|
| 13.50          | 15.71         | +2.21 | 39.69          | 42.50         | +2.81 | 55.89           | 59.80          |

GPU / 記憶體負擔

| 檢索方向             | 耗時 (秒) | PyTorch 峰值顯存 (MB) | 檢索結束佔用 (MB) |
|------------------|--------|-------------------|-------------|
| S2T (PC → Text)  | 0.5803 | 1006.8            | 685.3       |
| T2S (Text → PC)  | 0.5603 | 1006.8            | 685.3       |
| S2I (PC → Image) | 0.5625 | 1006.8            | 685.3       |
| I2S (Image → PC) | 0.5614 | 1006.8            | 685.3       |

+RAG

| 檢索方向            | 耗時 (秒) | PyTorch 峰值顯存 (MB) | 檢索結束佔用 (MB) |
|-----------------|--------|-------------------|-------------|
| T2S (Text → PC) | 0.6212 | 1006.8            | 685.3       |

| Baseline↵    | RR@1↵    |               | RR@5↵    |               | NDCG↵    |               |
|--------------|----------|---------------|----------|---------------|----------|---------------|
|              | w/o Rag↵ | w Rag↵        | w/o Rag↵ | w Rag↵        | w/o Rag↵ | w Rag↵        |
| Parts2Words↵ | 12.72↵   | 13.65↵        | 32.98↵   | 35.15↵        | 23.13↵   | 24.51↵        |
| TriCoLo↵     | 10.25↵   | 11.42↵        | 29.07↵   | 32.28↵        | 19.85↵   | 21.25↵        |
| Ulip↵        | 3.23↵    | 8.5↵          | 7.33↵    | 28.3↵         | 9.45↵    | 18.64↵        |
| Ulip-2↵      | 13.50↵   | <b>17.50↵</b> | 39.69↵   | <b>42.99↵</b> | 26.89↵   | <b>29.50↵</b> |

| Retrieval Direction↵ | Time (s)↵ | Memory (MB)↵ |
|----------------------|-----------|--------------|
| ULIP↵                | 0.5603↵   | 1006.8↵      |
| ULIP+RAG↵            | 0.6212↵   | 1238.34↵     |

| RAG Component↵                      | Time (s)↵ | Δ Memory (MB)↵ |
|-------------------------------------|-----------|----------------|
| Dense Retriever Encoder↵            | 0.0430↵   | 58.71↵         |
| Corpus Search ↵<br>(FAISS IVFFlat)↵ | 0.0009↵   | 94.38↵         |
| Cross-Fusion Module↵                | 0.0180↵   | 78.45↵         |